

Projekt WŚP 2025

Algorytm wyrównywania histogramu (Histogram Equalization) na GPU z wykorzystaniem CUDA C++

Adam Błażejowski **198344**

Maciej Domeradski **197818**

1. Opis algorytmu

Wybrany przez nas algorytmem jest **wyrównywanie histogramu (Histogram Equalization)**, dostępny w OpenCV jako `cv::equalizeHist`. Proces ten składa się z trzech głównych etapów:

1. **Obliczenie histogramu:** Zliczenie wystąpień każdego z 256 poziomów jasności w obrazie.
2. **Obliczenie dystrybuanty (CDF):** Wykonanie sumy prefiksowej (`inclusive_scan`) na histogramie.
3. **Aplikacja tablicy LUT:** Przekształcenie każdego piksela obrazu wejściowego na podstawie obliczonej dystrybuanty w celu zwiększenia kontrastu.

2. Implementacja i wykorzystane techniki

2.1. Modern C++

- **Functors:** Zdefiniowano strukturę `struct Normalize`, która pełni rolę funktora operującego na danych podczas transformacji.
- **Containers:** Wykorzystano kontenery `thrust::device_vector` oraz `thrust::host_vector` do zarządzania pamięcią na GPU i CPU.
- **Type Aliases / Auto:** W kodzie wykorzystano inferencję typów (`auto`) przy pracy z iteratorami i pomiarze czasu, co zwiększa czytelność kodu.

2.2. Biblioteka Thrust

- **Algorytmy:**
 - **`thrust::inclusive_scan`** – użyte do obliczenia dystrybuanty (CDF) z histogramu.

- **thrust::minmax_element** – użyte do znalezienia minimalnej wartości w CDF.
- **thrust::copy** – użyte do transferu danych oraz materializacji wyników z iteratorów.
- **Fancy Iterators:**
 - **thrust::make_transform_iterator** – wykorzystany do obliczania wartości tablicy LUT "w locie" na podstawie dystrybucyj, bez konieczności alokowania bufora pośredniego dla znormalizowanych wartości przed skopiowaniem ich do finalnej tablicy LUT.
- **Vocabulary Types:**
 - **thrust::pair** – wykorzystany jako typ zwracany przez funkcję `thrust::minmax_element` (zawiera iteratory do minimum i maksimum).

2.3. Asynchroniczność, CUB i narzędzia NVIDIA

- **CUB vs Thrust/Custom:** Zaimplementowano funkcję `histogram_cub` wykorzystującą `cub::DeviceHistogram::HistogramEven`. Pozwoliło to na bezpośrednie porównanie wydajności bibliotecznej implementacji z własnym kernelem.
- **Synchronizacja:** Użyto `cudaDeviceSynchronize()` przed i po blokach pomiarowych, aby zapewnić, że mierzymy faktyczny czas wykonania operacji GPU, a nie tylko czas zlecenia zadań do kolejki.
- **NVTX:** Kod został zinstrumentowany znacznikami `nvtxRangePush` i `nvtxRangePop`, co pozwoliło na precyzyjną analizę poszczególnych etapów algorytmu w Nsight Systems.

2.4. Własne Kernele CUDA

- **Kernel histogramu (`histogram_kernel`):** Implementuje obliczanie histogramu z wykorzystaniem pamięci współdzielonej (Shared Memory). Każdy wątek inicjalizuje część prywatnego histogramu w bloku, wykonuje `atomicAdd` w pamięci shared (co jest znacznie szybsze niż w pamięci globalnej), synchronizuje się barierą `__syncthreads()`, a następnie sumuje wyniki do pamięci globalnej.
- **Kernel LUT (`lut_kernel`):** Prosty kernel mapujący piksele 1:1, demonstrujący wykorzystanie siatki wątków (`grid/block`) do przetwarzania obrazu.

3. Analiza wydajności

Testy przeprowadzono na obrazie o rozdzielczości 2304x1856. Pomiary czasu obejmują samą operację przetwarzania (bez czasu alokacji pamięci i wczytywania pliku, które są jednorazowe).



Porównanie zdjęcia wejściowego ze zdjęciem wynikowym działania algorytmu.

Tabela wyników (na podstawie logów aplikacji):

Operacja	Czas wykonania [ms]	Przyspieszenie względem CPU
OpenCV CPU (cv::equalizeHist)	6.99076 ms	1.0x (Bazowe)
CUDA Kernel	0.429198 ms	16.3x
CUB Histogram	0.176948 ms	38.8x

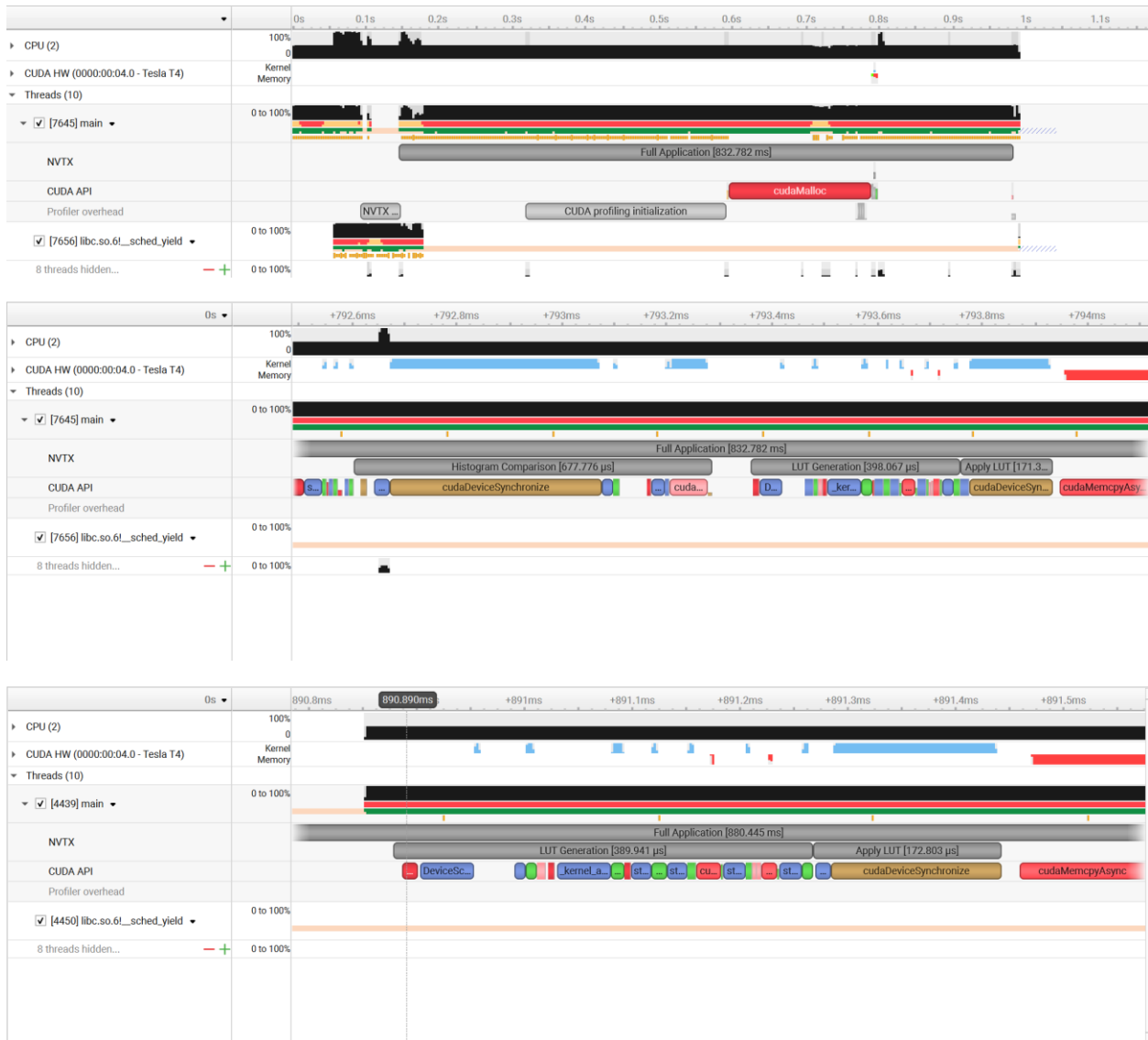
Wnioski:

- 1. Dominacja GPU:** Obie implementacje na GPU są o rzędy wielkości szybsze od referencyjnej implementacji CPU z biblioteki OpenCV.
- 2. Histogram Kernel vs CUB:** W tym konkretnym przypadku własny kernel (histogram_kernel) osiągnął czas 0.44 ms, podczas gdy implementacja CUB osiągnęła 0.18 ms. Wynika to prawdopodobnie z wysoce zoptymalizowanych algorytmów redukcji w CUB, które lepiej wykorzystują przepustowość pamięci w architekturze Turing (Tesla T4) niż prosta implementacja na Shared Memory.

3. **Koszty pamięci:** Czasy w tabeli dotyczą samych obliczeń. Jak pokazuje profilowanie (poniżej), transfer pamięci Host-Device zajmuje więcej czasu niż samo obliczenie histogramu.

4. Profilowanie - Nsight Systems

Analiza została przeprowadzona przy użyciu narzędzia Nsight Systems z wykorzystaniem znaczników **NVTX**.



Szczegółowa analiza profilu:

1. Analiza NVTX Range Statistics:

- Cała aplikacja ("Full Application") trwała ok. 832 ms, jednak większość tego czasu to inicjalizacja kontekstu CUDA i alokacje pamięci.
- **Kluczowe etapy algorytmu (GPU):**
 - **Histogram Comparison:** 677 μ s (obejmuje oba warianty histogramu i synchronizację).
 - **LUT Generation:** 398 μ s (Scan + MinMax + Transform).
 - **Apply LUT:** 171 μ s (Zastosowanie tablicy na obrazie).

2. Analiza API i Kerneli (CUDA Statistics):

- **Najbardziej czasochłonna operacja API:** cudaMalloc (9 wywołań) zajęło łącznie aż 197 ms. Potwierdza to zasadę, że alokacja pamięci na GPU jest kosztowna i powinna być wykonywana raz (np. na początku programu).
- **Wydajność Kerneli:**
 - histogram_ kernel: zajął średnio 396 μ s.
 - lut_kernel: zajął średnio 150 μ s.
 - Dla porównania kernel CUB (DeviceHistogramSweepKernel) zajął 67 μ s dla samej fazy "sweep", ale towarzyszyły mu inne kernele pomocnicze (np. DeviceHistogramInitKernel), co sumarycznie złożyło się na dłuższy czas wykonania całego API CUB.

3. Transfer Pamięci (Memory Throughput):

- Kopiowanie danych z hosta na urządzenie (HtoD) zajęło 867 μ s, a z urządzenia na hosta (DtoH) średnio 2.13 ms.
- **Wniosek:** Czas transferu danych (ok. 2.98 ms łącznie) jest dłuższy niż czas obliczeń własnego kernela (0.42 ms) i aplikacji LUT (0.17 ms) razem wziętych. Wskazuje to, że dla pojedynczego obrazu algorytm jest ograniczony przepustowością pamięci (memory-bound) na poziomie szyny PCIe.

5. Wnioski projektowe

1. **Shared Memory:** Implementacja histogramu na Global memory byłaby drastycznie wolniejsza. Zastosowanie Shared Memory i redukcja wyników wewnątrz bloku, jak w zaimplementowanym *histogram_kernel*.
2. **Thrust:** Wykorzystanie `thrust::inclusive_scan` i `thrust::make_transform_iterator` pozwoliło zaimplementować skomplikowaną logikę (CDF, normalizacja) w zaledwie kilku liniach kodu, zachowując wysoką czytelność i wydajność.
3. **Bottleneck:** Profilowanie wykazało, że przy przetwarzaniu pojedynczego obrazu największym narzutem jest alokacja pamięci oraz transfer danych. Aby w pełni wykorzystać potencjał GPU w systemie czasu rzeczywistego, należałoby przeprowadzić operacje na większym zbiorze danych – większej ilości zdjęć/ zdjęć o większej rozdzielczości.
4. **Poprawność wyników:** Otrzymany obraz wynikowy `wynik_gpu.png` cechuje się poprawnie wyrównanym kontrastem, porównywalnym (nierozróżnialny gołym okiem) z wynikiem referencyjnym OpenCV, co potwierdza poprawność implementacji algorytmu Histogram Equalization.